MANIPAL UNIVERSITY JAIPUR
University u der Sectio 2(f) of the UGC Act

NAME	**HIRDYANSH VARSHNEY**
ROLL NO.	**2414500172**
PROGRAMME	**BCA**
SEMESTER	**4th**
SUBJECT NAME	**Software Engineering**
SUBJECT CODE	**DCA2206**

SET – I

Question [1]. Define a system. Explain its elements and characteristics. Discuss different types of systems with suitable examples.

- **Definition of a System**

A system is an organized group of components that work together toward a shared goal. No single component can achieve the goal alone — it is the combination and coordination of all parts that makes the system work. The word 'system' comes from the Greek word systema meaning an organized whole.

For example, a railway booking system includes passengers, booking counters, databases, payment gateways, and ticket printers. Each part has its role, but together they make it possible to reserve a seat on a train. In software engineering, the term system usually refers to an information system — a set of components that collect data, process it, store it, and produce useful information for decision making.

➢ Elements of a System

Every system is built from the following elements. Input is whatever enters the system from the outside — for example, a passenger entering their travel details on a booking website. Process transforms the input into something useful — the server checks availability and calculates the fare. Output is the result — a confirmed ticket with seat number. Feedback is information about the output sent back to the system for monitoring and correction — for example, if payment fails, an error message is shown so the user can try again. Control compares the actual output with the expected output and triggers corrections if needed. Boundary defines what is inside the system and what is outside. Environment is everything outside the boundary that can still affect the system — like internet connectivity or bank server status. Interface is where two components or systems connect and exchange data — like the payment gateway connecting the booking system to the bank.

➢ Characteristics and Types of Systems

Key characteristics of a system include: having a clear purpose, being structured, being interconnected (one part affects others), being hierarchical (large systems contain subsystems), and self-regulating — seeking stability when disturbed, a property called homeostasis.

Physical vs Abstract Systems: Physical systems have real, tangible parts like machines and people. Abstract systems exist only as concepts, like a mathematical model or a set of business rules. For example, a payroll software is a physical system, while the salary calculation formula used inside it is abstract.

Open vs Closed Systems: An open system exchanges resources or information with its environment. A supermarket is an open system — it gets goods from suppliers and money from customers. A closed system does not interact with its environment. True closed systems are very rare. A thermally insulated flask is a rough example.

Deterministic vs Probabilistic Systems: A deterministic system always gives the same output for the same input — a simple interest calculator always gives the same result for the same values. A probabilistic system gives uncertain outputs — a recommendation engine may suggest different products to the same user at different times based on trends and user behavior.

Real-Time vs Batch Systems: Real-time systems respond immediately — like an online payment system that confirms a transaction within seconds. Batch systems process collected data at a scheduled time — like a bank that calculates daily account interest at midnight every night.

Question [2]. Define Software Engineering and explain its need. Discuss software characteristics. Explain McCall's Quality Factors in detail.

➢ Software Engineering – Definition and Need

Software engineering is the disciplined and systematic application of engineering concepts to the design, development, testing, and maintenance of software. Just like civil engineers follow standards and methods to build bridges, software engineers follow structured processes to build reliable software systems.

The need for software engineering arose from the 'software crisis' of the 1960s and 1970s. As computers became more powerful, software projects grew very large and complex. But without any formal process, these projects kept failing — they ran over budget, missed deadlines, had too many bugs, or did not satisfy users at all. It became clear that writing code informally was not enough for large, critical systems. Software engineering brought in structured methodologies, tools, and quality practices to solve this. Today, software controls aircraft navigation, hospital life-support machines, banking systems, and national power grids. Failures in such systems can cost lives and huge money — which is why proper engineering is not optional but essential.

➢ Software Characteristics

Software is different from other engineered products in several important ways. It is intangible — you cannot physically measure or weigh it. It does not wear out with use like a machine does, but it can become outdated when the environment or user needs around it change. Software is highly complex — even a simple mobile app can have hundreds of thousands of lines of code with countless interactions that are difficult to fully predict or control. Software is easily modifiable — changes can be made quickly, but unplanned changes cause 'software rot' where the code gradually becomes messy and hard to manage. Software development is purely a human intellectual activity — creativity, skill, and careful thinking drive the process. Finally, software defects do not develop slowly; a single bug can cause a total and sudden system failure without any prior warning signs.

➢ McCall's Quality Factors

Proposed by Jim McCall in 1977, this model defines software quality through 11 factors grouped under three perspectives — how the software is used, how it is changed, and how it is moved to new environments.

Product Operation factors (day-to-day use): Correctness — does the software produce accurate results as per specifications? Reliability — does it run without crashing or failing regularly? Efficiency — does it perform well without wasting CPU or memory? Integrity — does it prevent unauthorized access to sensitive data? Usability — can users learn and work with it without much difficulty?

Product Revision factors (making changes): Maintainability — how easily can existing bugs be found and fixed? Flexibility — how easily can the software be modified to add new features? Testability — how easily can the software be re-tested after changes to confirm it still works?

Product Transition factors (moving to new environments): Portability — how easily can the software run on different hardware or operating systems? Reusability — can modules from this software be used in other future projects? Interoperability — can the software exchange data and work alongside other systems smoothly?

McCall's model is valuable because it shifts the view of quality beyond just 'does it work' to include long-term concerns like maintainability and adaptability. It helps teams build software that stays useful and manageable for years after delivery.

Question [3]. What is Requirement Analysis? Explain requirement anticipation and the role of a system analyst. Discuss the knowledge and qualities required for a good system analyst.

➢ Requirement Analysis

Requirement analysis is the process of carefully studying, understanding, and documenting what a software system must do before its design and development begin. It is the foundation of the entire project — if requirements are wrong or incomplete, even a perfectly written codebase will fail to satisfy users.

The process involves gathering information from clients and users through interviews, questionnaires, workshops, and observation of existing processes. These requirements are analyzed and organized into a Software Requirements Specification document, or SRS, which serves as a formal agreement between the client and development team. Requirements are divided into two categories: functional requirements, which describe what the system should do (for example, 'the system must send an email confirmation after every booking'), and non-functional requirements, which describe how the system should perform (for example, 'the system must handle 500 simultaneous users without slowing down').

➢ Requirement Anticipation and Role of System Analyst

Requirement anticipation means identifying needs that users did not explicitly state but which are obviously necessary for a complete system. Users naturally focus on their main tasks and often leave out smaller but essential details. A good analyst uses experience and domain knowledge to spot and fill these gaps proactively.

For example, a grocery chain requests a stock management system. The client asks for features to track incoming stock and mark items as sold. But an experienced analyst would also anticipate needs like automatic low-stock alerts, expiry date tracking, supplier contact management, and monthly consumption reports. None of these were mentioned, but all are clearly needed for a useful stock management system.

The system analyst is the connecting link between the client and the technical team. Their work includes conducting meetings and workshops to gather requirements, documenting the current system's strengths and weaknesses, preparing the SRS and system models, creating UML and data flow diagrams, coordinating with developers throughout the project, and finally verifying that the delivered system matches the original requirements. A skilled analyst can save an entire project; a poor one can ruin it regardless of how good the developers are.

➢ Knowledge and Qualities of a Good System Analyst

Technical Knowledge: A good analyst must understand software development methodologies and the full software development lifecycle. They should know how to create system models using tools like UML, data flow diagrams, and entity-relationship diagrams. They need a working understanding of databases, networks, and basic programming — not to write code, but to know what is technically possible and to communicate clearly with the development team.

Domain Knowledge: The analyst must understand the business area they are working in. For example, an analyst building a logistics software must understand concepts like supply chain, freight management, and delivery tracking. Without this, they cannot judge whether the requirements they gather make practical business sense.

Communication Skills: This is arguably the most important quality. The analyst must listen carefully — often users struggle to explain what they really need, and the analyst must ask the right questions to draw out the true requirements. They must also write clearly — the SRS must be precise and unambiguous because even small unclear statements can lead to costly misunderstandings during development.

Analytical and Problem-Solving Skills: The analyst frequently encounters conflicting requirements from different stakeholders, incomplete information, and technical constraints. They need to break down complex problems, identify inconsistencies, and suggest practical solutions that satisfy everyone involved.

Patience and Adaptability: Clients often change their minds, stakeholders give contradictory feedback, and requirements evolve over time. The analyst must handle all of this calmly without losing focus. They must also adapt their working style to suit different clients and project situations. Finally, strong attention to detail is critical — a single missing or ambiguous requirement can create significant rework later in the project, costing time and money.

SET – II

Question [4]. Explain input and output design in software systems. Describe UML diagrams and their importance in system design with examples.

➢ Input Design

Input design is the process of planning how data will be entered into a software system. The quality of a system's output depends entirely on the quality of its input. If data entered is incorrect or incomplete, all processing done on it will produce wrong results — this is known as 'garbage in, garbage out.'

Principles of good input design: Collect only the data the system truly needs — extra fields slow users down and increase mistakes. Organize input forms logically with clear labels and a natural flow. Build validation into the system — reject wrong data types, out-of-range values, or empty mandatory fields immediately with helpful error messages. Use dropdowns, checkboxes, and auto-fill wherever possible to reduce manual typing. For example, in a hospital patient registration form, the patient's date of birth should use a date picker instead of a free-text field to avoid invalid entries like '31-Feb-2000'.

➢ Output Design

Output design is the process of deciding what information the system will present, in what format, and to which users. Output is the most visible part of any system — it is what users actually interact with and use to make decisions, so it must be accurate, clear, and well-organized.

Principles of good output design: Only include relevant information — too much data overwhelms users and hides what is important. Use proper headings, column titles, and units of measurement. Design for the right audience — an operations manager needs summary charts and totals, while an auditor needs detailed transaction-by-transaction records. Ensure timeliness — a daily attendance report must be ready before the workday begins, not at the end. Also consider the output format — some outputs are printed reports, some are on-screen dashboards, and some are automated emails or SMS messages. Each format needs different layout and design considerations.

➢ UML Diagrams and Importance

UML stands for Unified Modeling Language. It is a standardized visual language for representing the structure and behavior of software systems. UML uses diagrams to communicate the system design clearly to developers, testers, clients, and other stakeholders. It was standardized by the Object Management Group and is used industry-wide as the professional standard for software design documentation.

Class Diagram: Shows the classes of the system, their attributes and methods, and the relationships between them. It is the most important structural diagram in object-oriented design. For example, in a hotel booking system, classes could be Room, Guest, Booking, and Invoice. The Room class has attributes like room number, type, and rate. The Guest class has name and contact details. The Booking class links a Guest to a Room with check-in and check-out dates. Relationships such as 'one Guest can have many Bookings' are clearly shown. Developers use this to write the actual classes in code.

Use Case Diagram: Represents the system from the user's perspective, showing what the system does (use cases) and who uses it (actors). For example, in a mobile banking application, actors are Customer and Bank Employee. Use cases include Login, Check Balance, Transfer Funds, Pay Bill, and Download Statement. This diagram is extremely useful in requirement gathering meetings because even non-technical clients can understand and validate it easily.

Sequence Diagram: Shows the order in which objects communicate with each other over time to complete a task. For example, a sequence diagram for fund transfer would show: Customer sends transfer request → Application verifies login → Application checks balance in Database → if sufficient, Database deducts amount → Application credits recipient account → Application sends SMS confirmation to Customer. This helps developers understand the exact flow and order of operations needed.

Activity Diagram: Similar to a flowchart — it shows the step-by-step flow of activities in a process, including decisions and parallel paths. For example, an activity diagram for an online food order would show: Browse Menu → Add Items to Cart → Review Order → Enter Address → Choose Payment → Place Order → if payment success then Confirm Order, else Show Payment Failed.

Component and Deployment Diagrams: Component diagrams show how different software modules depend on each other. Deployment diagrams show which software components run on which physical hardware. These are used by system architects when planning how to deploy a large application across multiple servers.

Importance of UML: UML diagrams replace lengthy written descriptions with visual blueprints that are faster to understand. They provide a common language for technical and non-technical

team members. They help in spotting design problems early — before any code is written, when they are cheapest to fix. They also serve as official project documentation that is useful during future maintenance and onboarding of new team members.

Question [5]. Define software testing and explain its characteristics. Differentiate between verification and validation with suitable examples.

➢ Software Testing – Definition and Characteristics

Software testing is the process of executing a software program with the purpose of finding defects or confirming that it works correctly as per its requirements. Testing is not the same as simply running the software — it is a planned activity with specific test inputs, expected outputs, and systematic observation of actual results.

Key characteristics of software testing are as follows. Testing is intentional — unlike a user who runs software to use it, a tester runs software specifically to find what is wrong with it. Testing cannot be exhaustive — for any real software, the number of possible inputs, paths, and scenarios is practically infinite, so testing must prioritize the highest-risk areas. Testing must start early — reviewing requirements and design documents for errors is also a form of testing, and catching defects at these stages costs far less than catching them after development. Tests must be repeatable — the same test should always produce the same result under the same conditions, especially during regression testing where old tests are re-run after changes to check nothing is broken. Testing is best done by an independent team — developers unconsciously avoid testing their own code in ways that would prove it wrong, so an independent tester brings a fresh and more critical perspective. All testing activities must be documented — what was tested, what inputs were used, what was expected, and what actually happened. Finally, testing shows that defects exist but cannot prove their complete absence — even after thorough testing, there may be undiscovered bugs.

➢ Verification vs Validation

Verification and validation are both quality assurance activities but they serve very different purposes and are done in different ways.

Verification asks: 'Are we building the product correctly?' It checks that the software is being developed according to defined specifications, standards, and design documents at every stage of the development process. Verification does not require running the software — it works through reviews, inspections, walkthroughs, and static code analysis. Verification activities happen throughout development, not just at the end.

Common examples of verification include: a requirements review where the team checks if the SRS document clearly covers all client needs without contradictions; a design review where the system design is checked against the requirements; a code inspection where senior developers read through code to verify it correctly implements the design and follows coding standards; and checking that every requirement in the SRS has at least one corresponding test case in the test plan.

Validation asks: 'Are we building the correct product?' It checks that the finished software actually solves the user's real-world problem and meets their actual needs. Validation requires

running the completed software and testing it in realistic conditions. Validation happens at the end of development or at the end of each development cycle. The most common form of validation is User Acceptance Testing (UAT), where real end users test the software in a real or simulated work environment.

A practical example to show the difference clearly: A logistics company commissions a delivery tracking software. The development team works carefully — they verify the design against the SRS, do code reviews for each module, and run hundreds of unit tests. Every verification check passes. But when drivers and delivery managers use the software during UAT, they find that the system tracks deliveries city-by-city but cannot handle inter-state deliveries that cross through multiple states. This feature was never written in the SRS because no one thought to ask. The software was built exactly as specified — so verification succeeded — but it does not solve the company's actual problem — so validation failed.

This example shows why both are needed separately. Verification ensures the team is following the right process and building to spec. Validation ensures the spec itself was correct and that the final product is genuinely useful. Skipping either one creates different types of problems: skipping verification leads to messy code full of hidden bugs; skipping validation leads to a technically clean system that nobody actually wants to use.

Question [6]. Explain Black Box Testing and White Box Testing. Discuss equivalence partitioning, boundary value analysis, and statement & branch coverage techniques.

➢ Black Box Testing

Black box testing is a testing approach where the tester evaluates the software based purely on its inputs and outputs, without any knowledge of its internal code or logic. The software is treated as a sealed box — the tester knows nothing about what is inside. Test cases are designed entirely from the user's perspective, based on the system's requirements and expected behavior.

Black box testing is effective at finding functional gaps — situations where the software does not do what it is supposed to from a user's point of view. It is performed by a QA team or end users who were not involved in development. Since these testers have no assumptions about the internal logic, they often test in ways that developers would not think to test. Common black box techniques include equivalence partitioning and boundary value analysis.

➢ White Box Testing

White box testing, also called glass box or structural testing, is a testing approach where the tester has complete knowledge of and access to the software's internal source code, logic, and structure. Test cases are designed to cover specific paths, branches, loops, and conditions within the actual code.

White box testing is typically done by developers during unit testing and integration testing. It is effective at finding logical errors buried in the code, identifying unreachable code that can never be executed, and confirming that all important code paths are covered by tests. Common white box techniques include statement coverage and branch coverage. White box and black

box testing complement each other — black box catches 'what is wrong from outside' and white box catches 'what is wrong inside'.

➢ Testing Techniques

Equivalence Partitioning is a black box technique. The idea is that all possible inputs to a system can be divided into groups called equivalence partitions, where every value within a group is expected to behave identically. Testing one value from each group is enough — if it works for that value, it should work for all others in the same group. This dramatically reduces the number of test cases needed without losing coverage quality.

For example, an electricity bill calculator accepts monthly units consumed. Valid input is 1 to 9999 units. We form three partitions: below 1 or zero/negative (invalid), 1 to 9999 (valid), above 9999 (invalid). We test one value from each: 0 (rejected), 500 (accepted and calculated), 10500 (rejected). Instead of testing thousands of values, just three representative tests are enough to cover all partitions.

Boundary Value Analysis is a black box technique that goes further than equivalence partitioning by specifically targeting the boundary edges of partitions. Programming errors are most likely to occur at boundaries — for example, using greater than instead of greater than or equal to in a condition. Boundary value analysis tests the value just below the boundary, the exact boundary, and the value just above it for each boundary in the input range.

Using the same units example (valid: 1 to 9999), boundary value analysis tests: 0 and -1 at the lower edge, 1 and 2 just inside, and 9998 and 9999 at the upper edge, and 10000 just outside. If a programmer wrote units > 1 instead of units >= 1, equivalence partitioning (testing 500) would not catch this but boundary value analysis (testing exactly 1) would catch it immediately. This is why boundary value analysis is considered more thorough than basic equivalence partitioning.

Statement Coverage is a white box technique that measures what percentage of the executable code statements have been run during testing. The formula is: (statements executed / total statements) × 100. For example, a function with 60 statements where only 48 are exercised by tests has 80% statement coverage. The goal is to reach 100% — meaning no line of code was completely skipped during testing. Statement coverage is the most basic form of white box coverage. Its limitation is that 100% statement coverage does not guarantee all decision paths are tested — you can execute every line once without ever hitting both sides of an if-else condition.

Branch Coverage is a stronger white box technique that ensures both possible outcomes of every decision point are tested. A decision point is any place in the code where execution can go in two directions — if-else conditions, while loops, for loops, switch cases, and ternary expressions. For each decision, the test suite must cover the case where the condition is true and the case where it is false. For example, in the code block if (temperature > 100) { raise_alarm() } else { log_normal() }, branch coverage requires one test where temperature exceeds 100 and one where it does not. Achieving 100% branch coverage guarantees 100% statement coverage as well, making it a stronger standard. In safety-critical software like medical devices or aviation systems, 100% branch coverage is often a regulatory requirement.